

BÁO CÁO THỰC HÀNH LAB 3

PIPES AND REDIRECTION

Người báo cáo: Liamz07

1. Mục tiêu bài thực hành:

- Thành thạo kỹ thuật điều hướng luồng dữ liệu chuẩn trong Linux: Standard Input (stdin - 0), Standard Output (stdout - 1) và Standard Error (stderr - 2).
- Hiểu và vận dụng chính xác các toán tử điều hướng ghi đè (>), ghi tiếp (>>), đọc dữ liệu (<), điều hướng luồng lỗi (2>), và gộp luồng xuất/lỗi (2>&1, &>).
- Sử dụng kết hợp toán tử ống dẫn (|) cùng các câu lệnh xử lý văn bản (cat, grep, wc, sort, uniq) để xây dựng chuỗi xử lý dữ liệu tự động.

2. Nội dung và phương pháp giải quyết chi tiết:

CHUẨN BỊ MÔI TRƯỜNG

- Trước khi làm bài tập, chạy chuỗi lệnh sau để tạo thư mục và các tập tin dữ liệu mẫu phục vụ cho việc thực hành:

```
mkdir -p ~/lab3_redirection
cd ~/lab3_redirection

# Tạo file log truy cập mẫu (access.log)
# Lưu ý: Chỗ này viết cách xuống dòng để dễ cap màn hình, trong thực tế bạn có thể viết liền một dòng
echo -e "192.168.1.10 - [09/Oct/2026] \"GET /index.html HTTP/1.1\" 200\n192.168.1.11 - [09/Oct/2026] \"GET /about.html HTTP/1.1\" 200\n192.168.1.12 - [09/Oct/2026] \"GET /favicon.ico HTTP/1.1\" 404\n192.168.1.13 - [09/Oct/2026] \"POST /login HTTP/1.1\" 500\n192.168.1.14 - [09/Oct/2026] \"GET /missing.php HTTP/1.1\" 404\n192.168.1.15 - [09/Oct/2026] \"GET /images/logo.png HTTP/1.1\" 404" > access.log

# Tạo danh sách dữ liệu để sắp xếp
echo -e "UserA\nUserC\nUserB\nUserA\nUserD" > users.txt
```

BÀI 1: ĐIỀU HƯỚNG ĐẦU RA CHUẨN (STANDARD OUTPUT REDIRECTION)

- Đề bài:**
 - Tạo tập tin system_info.txt, ghi kết quả thông tin hệ thống (uname -a) vào file, sau đó ghi đè dòng chữ "Updated Info" vào chính file đó.
 - Thêm thông tin ngày giờ hiện tại (date) vào cuối tập tin system_info.txt mà không làm mất nội dung cũ.
- Phương pháp giải quyết:**
 - Khởi tạo file rỗng bằng touch.
 - Sử dụng toán tử ghi đè > để điều hướng kết quả của lệnh uname -a vào file.
 - Tiếp tục dùng toán tử > kết hợp với echo để ghi đè nội dung mới, thay thế toàn bộ dữ liệu cũ.
 - Sử dụng toán tử ghi tiếp >> thay vì toán tử ghi đè >.
- Lệnh thực thi & Cú pháp tổng quát:**
 - Tạo tập tin system_info.txt, ghi kết quả thông tin hệ thống (uname -a) vào file, sau đó ghi đè dòng chữ "Updated Info" vào chính file đó.

- ❖ *Lệnh:* touch system_info.txt && uname -a > system_info.txt && echo "Updated Info" > system_info.txt
- ❖ *Cú pháp:* command > file_path
- ❖ *Giải thích:*
 - uname (Unix Name) -a (all): In toàn bộ thông tin cấu hình kernel/hệ thống.
 - echo "Updated Info": In ra chuỗi văn bản "Updated Info".
 - >: Toán tử điều hướng đầu ra chuẩn (stdout). Nếu file chưa tồn tại sẽ tự động tạo mới, nếu file đã có sẵn nội dung sẽ ghi đè hoàn toàn dữ liệu cũ.
 - system_info.txt: Tập tin đích nhận luồng dữ liệu.
- Thêm thông tin ngày giờ hiện tại (date) vào cuối tập tin system_info.txt mà không làm mất nội dung cũ.
 - ❖ *Lệnh:* date >> system_info.txt
 - ❖ *Cú pháp:* command >> file_path
 - ❖ *Giải thích:*
 - date: Lệnh xuất ra ngày, giờ hệ thống hiện tại.
 - >>: Toán tử điều hướng ghi tiếp (append) vào cuối tập tin mà giữ nguyên các dòng dữ liệu đang có trước đó.

BÀI 2: XỬ LÝ LOG VÀ KẾT NỐI LỆNH BẰNG ỐNG DẪN (PIPES - |)

- **Đề bài:**
 - Đọc file access.log và lọc ra tất cả các dòng chứa mã lỗi 404.
 - Đếm tổng số lượng phản hồi lỗi 404 xuất hiện trong tập tin access.log.
 - Đọc file users.txt, sắp xếp danh sách theo thứ tự bảng chữ cái và loại bỏ các tên người dùng bị lặp lại.
- **Phương pháp giải quyết:**
 - Dùng lệnh cat đọc toàn bộ nội dung file access.log, sử dụng toán tử ống dẫn | truyền dữ liệu đầu ra này làm đầu vào cho lệnh grep "404".
 - Mở rộng chuỗi ống dẫn bằng cách nối thêm lệnh wc -l vào sau lệnh grep để đếm tổng số dòng lọc được.
 - Đọc dữ liệu bằng cat, truyền qua sort để sắp xếp dữ liệu tăng dần (điều kiện bắt buộc trước khi dùng uniq), sau đó truyền qua uniq để lọc danh sách duy nhất.
- **Lệnh thực thi & Cú pháp tổng quát:**
 - Đọc file access.log và lọc ra tất cả các dòng chứa mã lỗi 404.
 - ❖ *Lệnh:* cat access.log | grep "404"
 - ❖ *Cú pháp:* command1 | command2
 - ❖ *Giải thích:*
 - cat access.log: Đọc và in dữ liệu tập tin access.log.
 - | (Pipe): Chuyển luồng đầu ra chuẩn (stdout) của lệnh bên trái làm luồng đầu vào chuẩn (stdin) cho lệnh bên phải.
 - grep "404": Lọc và hiển thị các dòng văn bản có chứa chuỗi "404".
 - Đếm tổng số lượng phản hồi lỗi 404 xuất hiện trong tập tin access.log.
 - ❖ *Lệnh:* cat access.log | grep "404" | wc -l
 - ❖ *Cú pháp:* command1 | command2 | command3
 - ❖ *Giải thích:*

- `wc` (Word Count): Lệnh đếm ký tự, từ, dòng.
- `-l` (Lines): Tùy chọn chỉ định lệnh `wc` đếm số lượng dòng dữ liệu nhận được từ ống dẫn.
- Đọc file `users.txt`, sắp xếp danh sách theo thứ tự bảng chữ cái và loại bỏ các tên người dùng bị lặp lại.
 - ❖ *Lệnh*: `cat users.txt | sort | uniq`
 - ❖ *Cú pháp*: `cat file | sort | uniq`
 - ❖ *Giải thích*:
 - `sort`: Lệnh sắp xếp các dòng văn bản theo thứ tự bảng chữ cái hoặc số.
 - `uniq`: Lệnh loại bỏ các dòng lặp lại nằm kề nhau.

BÀI 3: ĐIỀU HƯỚNG ĐẦU VÀO CHUẨN (STANDARD INPUT REDIRECTION)

- **Đề bài:** Sử dụng lệnh `wc -l` kết hợp toán tử `<` để đếm số dòng của file `access.log` thông qua luồng `stdin`.
- **Phương pháp giải quyết:** Dùng toán tử `<` để bơm dữ liệu trực tiếp từ file vào lệnh `wc -l` thay vì truyền file như một tham số đối số.
- **Lệnh thực thi & Cú pháp tổng quát:**
 - *Lệnh*: `wc -l < access.log`
 - *Cú pháp*: `command < file_path`
 - *Giải thích*:
 - ❖ `<`: Toán tử điều hướng đầu vào chuẩn (`stdin`). Lệnh phía trước sẽ nhận dữ liệu đọc từ tập tin phía sau thay vì chờ người dùng nhập từ bàn phím.

BÀI 4: ĐIỀU HƯỚNG LUỒNG BÁO LỖI (STANDARD ERROR REDIRECTION)

- **Đề bài:**
 - Chạy lệnh liệt kê một thư mục không tồn tại và bắt thông báo lỗi lưu vào tập tin `error.log`.
 - Kiểm tra đồng thời một file có thật (`access.log`) và một thư mục không có thật (`/non_existent_folder`), lưu cả kết quả thành công và thông báo lỗi vào chung một tập tin `combined_output.txt`.
- **Phương pháp giải quyết:**
 - Sử dụng toán tử `2>` (đổi hướng File Descriptor 2 - `stderr`).
 - Trong Linux có 2 luồng đọc lập là `stdout` (luồng 1) và `stderr` (luồng 2). Cần đổi hướng `stdout` vào file trước, sau đó dùng `2>&1` để nối luồng `stderr` nhập chung vào vị trí của `stdout`. Có thể dùng cú pháp gộp ngắn gọn `&>` trên các vỏ lệnh hiện đại như Bash/Zsh.
- **Lệnh thực thi & Cú pháp tổng quát:**
 - Chạy lệnh liệt kê một thư mục không tồn tại và bắt thông báo lỗi lưu vào tập tin `error.log`.
 - ❖ *Lệnh*: `ls /non_existent_folder 2> error.log`
 - ❖ *Cú pháp*: `command 2> error_file_path`
 - ❖ *Giải thích*:
 - `/non_existent_folder`: Thư mục không có trên hệ thống nhằm kích hoạt báo lỗi.
 - `2>`: Toán tử điều hướng luồng lỗi chuẩn (`stderr` - mã số 2) vào tập tin chỉ định. Khi dùng toán tử này, màn hình terminal sẽ hoàn toàn sạch sẽ, không hiển thị dòng lỗi.

- error.log: Tập tin lưu trữ thông báo lỗi.
- Kiểm tra đồng thời một file có thật (access.log) và một thư mục không có thật (/non_existent_folder), lưu cả kết quả thành công và thông báo lỗi vào chung một tập tin combined_output.txt.
 - ❖ *Lệnh*: `ls access.log /non_existent_folder > combined_output.txt 2>&1` (*Cú pháp chuẩn POSIX*) **hoặc** `ls access.log /non_existent_folder &> combined_output.txt` (*Cú pháp ngắn gọn Bash/Zsh*)
 - ❖ *Cú pháp*: `command > file_path 2>&1 ; command &> file_path`
 - ❖ *Giải thích*:
 - `> combined_output.txt`: Đẩy luồng thành công stdout (luồng 1) vào file.
 - `2>&1`: Chỉ định luồng lỗi stderr (luồng 2) hợp nhất vào cùng địa điểm với luồng 1 (&1).
 - `&>`: Toán tử đại diện gộp đồng thời cả luồng stdout và stderr đẩy thẳng vào tập tin chỉ định.

3. Kết luận và bài học rút ra:

- **Phân biệt bản chất các luồng dữ liệu:** Việc nắm vững 3 luồng dữ liệu stdin (0), stdout (1) và stderr (2) giúp quản lý và xử lý log hệ thống linh hoạt, ngăn chặn thông báo lỗi làm rác màn hình quản trị.
- **Nắm chắc thứ tự toán tử điều hướng:** Đã hiểu rõ nguyên lý hoạt động của `> file 2>&1` thay vì viết nhầm `2>&1 file` (khiến file bị hiểu sai thành một tham số đường dẫn của lệnh).
- **Sức mạnh của Pipes (|):** Kỹ thuật nối chuỗi câu lệnh cho phép giải quyết các bài toán phân tích log phức tạp một cách nhanh chóng chỉ bằng một dòng lệnh duy nhất trên Linux CLI.

HẾT!